Lab 5 – Support Vector Machine

1. Suppose that the following are a set of points in two classes:

$$\text{class 1} \quad : \quad \begin{pmatrix} 1 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} \tag{8.32}$$

$$\text{class 2} \quad : \quad \begin{pmatrix} 0 \\ 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} \tag{8.33}$$

Plot them and find the optimal separating line. What are the support vectors, and what is the margin?

Answer 1 – Optimal separating line, support vectors and margin

Code: task1_svm_linear.py → task1_result.png. A hard-margin linear SVM ($C = 10^6$) was fitted to the six points.

Optimal separating line

$$\mathbf{2x_1 + 2x_2 - 3 = 0} \qquad \textbf{i.e.} \qquad \mathbf{x_1 + x_2 = 1.5}$$

so w = (2, 2) and b = −3.

Support vectors

These are the three points lying exactly on the margin lines:

Point	Class	w · x + b
(1, 1)	+1	+1
(1, 0)	−1	−1
(0, 1)	−1	−1

The remaining points (1, 2), (2, 1) and (0, 0) give +3, +3 and −3. They lie beyond the margin and have no influence on the solution — removing them from the training set produces the identical line.

Margin

$$\mathbf{M = 2 / \|w\| = 2 / (2\sqrt{2}) = 1/\sqrt{2} \approx 0.7071}$$

The distance from the boundary to the nearest point is half of this, ≈ 0.3536.

Verification by hand

The closest approach between the two classes is between (1, 1) and the segment joining (1, 0) and (0, 1); the nearest point on that segment is its midpoint (0.5, 0.5), at a distance of $1/\sqrt{2} \approx 0.7071$. The

maximum-margin boundary is the perpendicular bisector of this shortest connection: it passes through (0.75, 0.75) with direction w ∝ (1, 1), which gives $x_1 + x_2 = 1.5$ — matching the SVM output exactly.

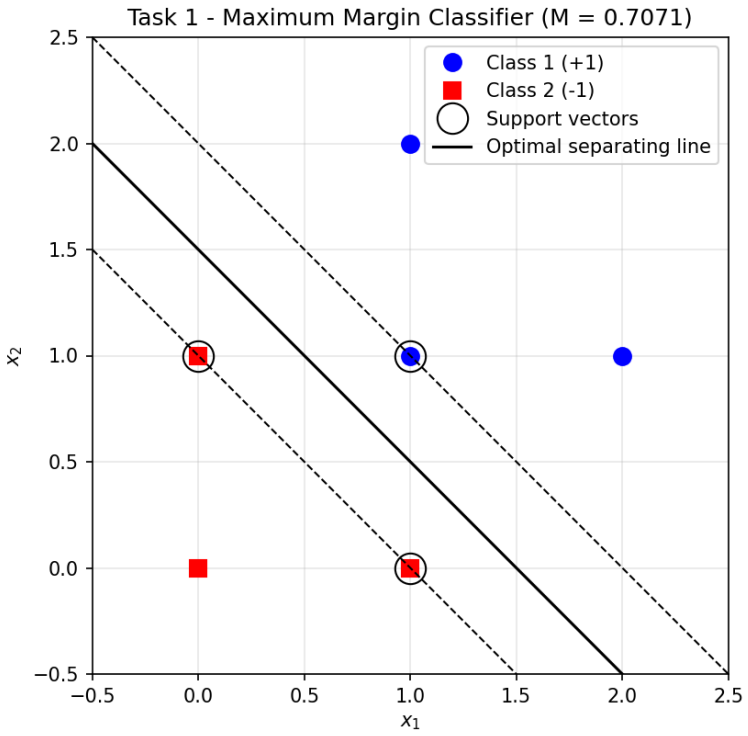


Figure 1 – Maximum margin classifier. Support vectors are circled; the dashed lines are the margin boundaries.

2. Use Perceptron code to classify the data in #1. Compare the classification line with the corresponding classification line that you obtained with SVM in problem 1. Explain the differences between the classification line and corresponding M's for both cases.

Answer 2 – Perceptron compared with the SVM

Code: task2_perceptron.py → task2_result.png. The perceptron (Marsland, Chapter 3; η = 0.25, 20 iterations) was trained five times from different random initial weights.

Run	Line found	Errors	Margin M
0	$0.5049x_1 + 0.2715x_2 - 0.7603 = 0$	0	0.0563
1	$0.4917x_1 + 0.5220x_2 - 0.7000 = 0$	0	0.4964
2	$0.4936x_1 + 0.4526x_2 - 0.5050 = 0$	0	0.0339
3	$0.5051x_1 + 0.5208x_2 - 0.7291 = 0$	0	0.5742
4	$0.0467x_1 + 0.0047x_2 - 0.0473 = 0$	0	0.0241
SVM	**$2x_1 + 2x_2 - 3 = 0$**	**0**	**0.7071**

Difference between the classification lines

The perceptron's line is not unique. All five runs classify every training point correctly, yet each one settles on a different boundary. The SVM returns the same line on every run, whatever the initialisation.

The reason is that the two algorithms optimise different objectives:

- The perceptron only minimises misclassifications. Its weight update is driven by the error term (y − t), so once every point is on the correct side the error is zero, the weights stop changing and training halts. Any one of the infinitely many separating lines is an acceptable stopping point, and which one is reached depends on the random initial weights, the learning rate and the order of the data.
- The SVM minimises $\frac{1}{2}\|w\|^2$ subject to $y_i(w \cdot x_i + b) \geq 1$. Zero training error is only the constraint; the quantity actually being maximised is the margin. This is a convex quadratic programme with a single global optimum, so the solution is unique.

Difference between the margins M

Every perceptron margin (0.0241 to 0.5742) is smaller than the SVM margin of 0.7071, and none can ever exceed it, because 0.7071 is the maximum attainable for this data.

This is what matters for generalisation. The boundary from run 4 passes within 0.012 of a training point, so a small amount of noise in a new test point would flip its predicted label. The SVM deliberately places the boundary as far from both classes as the geometry allows, which makes it considerably more robust even though both classifiers score identically on the training set.

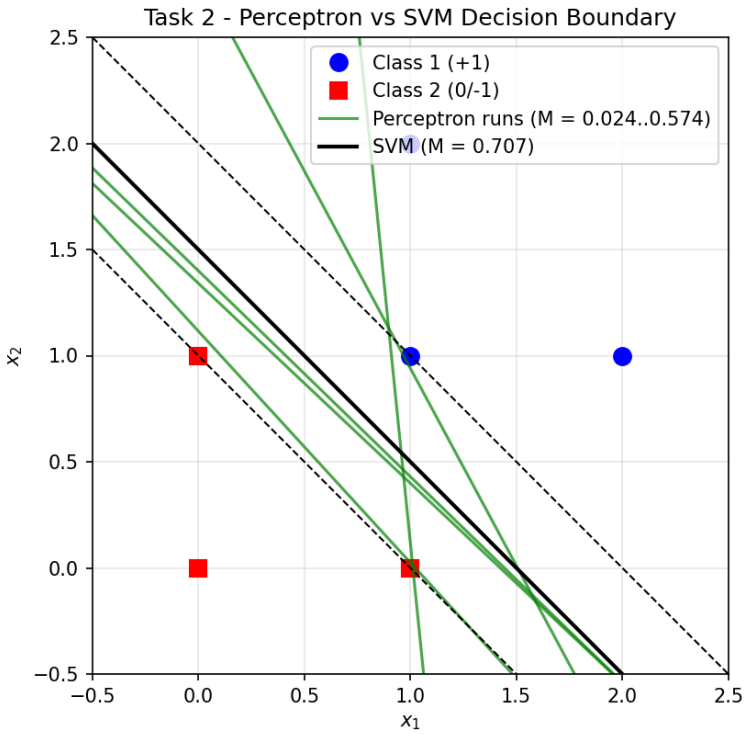


Figure 2 – Five perceptron boundaries (green) against the unique SVM boundary (black) with its margin.

3. Now consider the following non-linear labeled data:

Positive labels –

$$\left\{\begin{pmatrix}2\\2\end{pmatrix},\begin{pmatrix}2\\-2\end{pmatrix},\begin{pmatrix}-2\\-2\end{pmatrix},\begin{pmatrix}-2\\2\end{pmatrix}\right\}$$

Negative Labels

$$\left\{\begin{pmatrix}1\\1\end{pmatrix},\begin{pmatrix}1\\-1\end{pmatrix},\begin{pmatrix}-1\\-1\end{pmatrix},\begin{pmatrix}-1\\1\end{pmatrix}\right\}$$

Now use the following transformation function to transform the data so that it becomes linear:

$$\Phi_1\begin{pmatrix}x_1\\x_2\end{pmatrix}=\begin{cases}\begin{pmatrix}4-x_2+|x_1-x_2|\\4-x_1+|x_1-x_2|\end{pmatrix} & \text{if } \sqrt{x_1^2+x_2^2}>2\\ \begin{pmatrix}x_1\\x_2\end{pmatrix} & \text{otherwise}\end{cases} \quad (1)$$

Show your transformed data and classify using straight lines.

Answer 3 – Transformed data and linear classification

Code: task3_transform.py → task3_result.png.

All four positive points have radius √8 ≈ 2.83 > 2, so the first branch of Φ_1 applies. The negative points have radius √2 ≈ 1.41 ≤ 2 and are left unchanged.

Transformed data

Original (positive)	$\lvert x_1 - x_2 \rvert$	Transformed
(2, 2)	0	(2, 2)
(2, −2)	4	(10, 6)
(−2, −2)	0	(6, 6)
(−2, 2)	4	(6, 10)

Original (negative)	Transformed
(1, 1)	(1, 1)
(1, −1)	(1, −1)
(−1, −1)	(−1, −1)
(−1, 1)	(−1, 1)

Classification with a straight line

$$z_1 + z_2 - 3 = 0 \qquad \text{i.e.} \qquad z_1 + z_2 = 3$$

Support vectors: (2, 2) from the positive class and (1, 1) from the negative class. Margin: M = √2 ≈ 1.4142.

Check: $z_1 + z_2$ evaluates to 4, 16, 12 and 16 for the positives (all > 3) and to 2, 0, −2 and 0 for the negatives (all < 3), so the line separates them correctly.

In the original space the positive points sit at the corners of a large square surrounding the negative points at the corners of a small one, so no straight line can separate them. After applying Φ_1 the problem becomes linearly separable. This is the principle behind SVM kernels: change the space rather than the classifier.

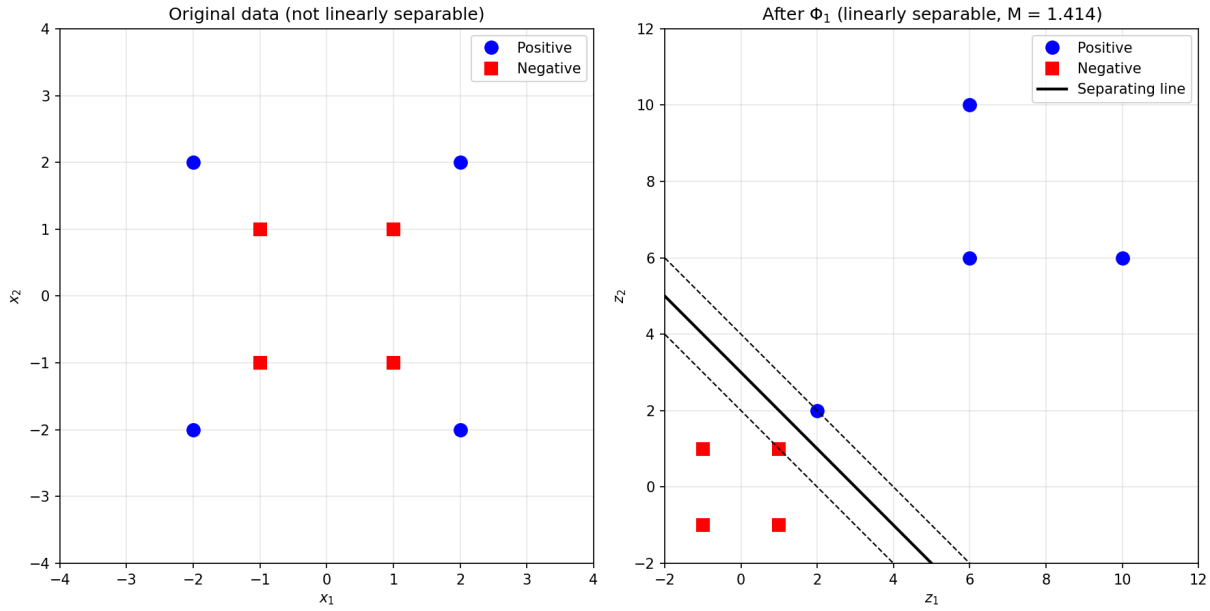


Figure 3 – The data before (left) and after (right) the transformation Φ_1.

4. Consider 2 circles – an inner circle surrounded by an outer circle. Obviously, they cannot be classified with SVM. However, you can apply the Kernel Trick (use a polynomial K) to convert the data into a higher dimensional space (3 in this case) – you can use np.dot etc to transform the data.

a. Use equations for an outer circle and make say 10 data points on the circle. Do the same for an inner circle.

b. Apply a polynomial Kernel to convert the data. Then plot the data.

Answer 4 – Two circles and the Kernel Trick

Code: task4_kernel_trick.py → task4_result.png.

Data

Ten points were generated on each circle using $x = (r\cos\theta, r\sin\theta)$ with θ evenly spaced over $[0, 2\pi)$: the inner circle with $r = 1$ (label -1) and the outer circle with $r = 3$ (label $+1$).

Polynomial kernel

The polynomial kernel $K(x, z) = (x \cdot z)^2$ has the three-dimensional feature map

$$\phi(x_1, x_2) = (x_1^2 ,\ \sqrt{2}\cdot x_1 x_2 ,\ x_2^2)$$

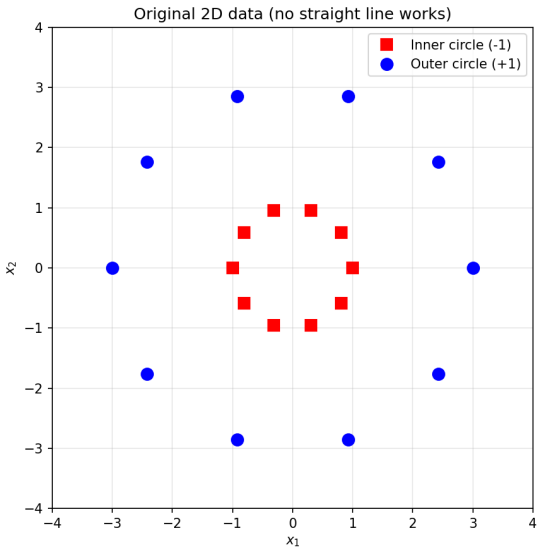
The script verifies the kernel identity numerically: $\max |K(x, z) - \phi(x) \cdot \phi(z)| = 2.8 \times 10^{-14}$, that is, equality to floating-point precision. This identity is exactly the kernel trick — the dot product in the higher-dimensional space can be computed without ever constructing the higher-dimensional vectors. For example $(1, 0) \mapsto (1, 0, 0)$ and $(3, 0) \mapsto (9, 0, 0)$.

Why this separates the two circles

In the new space the first and third coordinates always sum to the squared radius:

$$z_1 + z_3 = x_1^2 + x_2^2 = r^2$$

Every inner-circle point therefore lands on the plane $z_1 + z_3 = 1$ and every outer-circle point on the plane $z_1 + z_3 = 9$. What were two concentric rings in 2D become two parallel sheets in 3D, and any plane between them separates the classes.



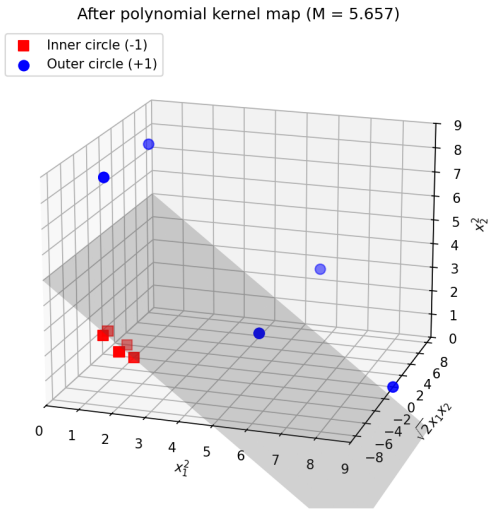


Figure 4 – The two circles in 2D (left) and after the polynomial kernel map, with the separating plane found by the SVM (right).

5. Run SVM on data in 4b (optional).

Answer 4b – SVM on the transformed data

A linear SVM fitted in the 3D feature space gives:

Separating plane	**$0.25z_1 + 0z_2 + 0.25z_3 - 1.25 = 0$**
Support vectors	8
Margin	M = 5.6569 = 4√2
Training accuracy	1.00

The coefficient of z_2 is zero, as expected: the $\sqrt{2}\cdot x_1x_2$ direction carries no information about which circle a point lies on, so the SVM ignores it. Dividing through by 0.25 gives

$$z_1 + z_3 = 5 \qquad \text{i.e.} \qquad x_1^2 + x_2^2 = 5$$

Mapped back to the original 2D space the plane becomes a circle of radius $\sqrt{5} \approx 2.236$ lying between the two rings — midway in squared radius between $r^2 = 1$ and $r^2 = 9$. A linear boundary in the feature space is therefore a non-linear (circular) boundary in the input space, which is precisely what was required.